Memoria técnica resumida — Nexo

Autor: Yerson Lasso
Módulo: Desarrollo de Interfaces (0488) · DAM
Fecha: 27 de octubre de 2026
Aplicación: https://sandbox.yerson.co/di/a1/

Objetivo y tecnologías

Nexo es una aplicación de gestión de usuarios/clientes y proyectos con dos roles: administrador y usuario estándar. Permite consultar y crear registros, actualizar proyectos, editar el perfil propio y exportar informes PDF y CSV. Utiliza datos sintéticos.

El frontend emplea **Angular 22.2.2, Ionic 9.0.7 y TypeScript 6.0.3**. El backend se ejecuta con **Node.js 26 y Express 5.3.0**. **Supabase Auth** gestiona la autenticación y **PostgreSQL** conserva los datos. **Terraform** administra el proyecto Supabase y la configuración de registro de cuentas.

Arquitectura e interfaz

La aplicación separa modelos tipados, servicios, componentes y plantillas HTML. Las páginas de Acceso, Resumen, Clientes, Proyectos, Mi cuenta y Ayuda se cargan bajo demanda mediante el router de Angular. Los guards protegen las rutas según la sesión y el rol.

Los componentes utilizan `signal` para el estado y `computed` para valores derivados. Los servicios `AuthService`, `DataService` y `ExportService` centralizan sesión, peticiones HTTP e informes. La tabla compartida `DataTableComponent` recibe datos mediante inputs y comunica acciones mediante outputs; proporciona búsqueda y ordenación.

Los formularios son reactivos y validan campos obligatorios, longitudes, correo y presupuesto. El alta comprueba de forma asíncrona si el correo ya existe. RxJS permite cargar clientes y proyectos conjuntamente y consultar la actividad cada cinco segundos. La interfaz incluye menú móvil, desplazamiento horizontal de tablas, etiquetas accesibles, foco visible y navegación por teclado.

Datos y seguridad

Usuarios y clientes comparten **`nexo_users`**, con exactamente **15 columnas**:

Identificación y perfil	Preferencias y auditoría
`id`, `email`, `full_name`, `display_name`, `role`	`locale`, `timezone`, `theme`, `notifications_enabled`
`phone`, `job_title`, `sector`	`created_at`, `updated_at`, `last_login_at`

La tabla de Clientes muestra todos los campos y el formulario permite introducir los datos y preferencias admitidos. ID, rol y fechas permanecen de solo lectura. Crear un cliente no crea automáticamente una cuenta de acceso.

`nexo_projects` relaciona cada proyecto con su usuario/cliente. PostgreSQL valida presupuesto, estado y progreso: Pendiente requiere 0 % y Completado requiere 100 %. `nexo_activity` guarda documentos JSONB generados por triggers dentro de la transacción de negocio. No se utiliza una segunda base NoSQL ni una suscripción en tiempo real.

Operación	Administrador	Usuario estándar
Consultar usuarios/clientes y proyectos	Todos	Solo su registro y sus proyectos
Editar perfil	Solo el propio	Solo el propio
Crear clientes y gestionar proyectos	Permitido	Denegado

Operación	Administrador	Usuario estándar
Consultar actividad global	Permitido	Sin acceso a sus registros

Las políticas **RLS** aplican los permisos en PostgreSQL. Express consulta Supabase con el JWT del usuario, sin utilizar la clave secreta para operaciones ordinarias. Los privilegios por columna impiden modificar roles y fechas desde el cliente. Los usuarios anónimos no tienen acceso a las tablas y el registro público de cuentas está desactivado.

La sesión utiliza una cookie HttpOnly, SameSite y Secure en producción, con duración máxima de una hora. Las credenciales y los tokens privados se guardan en `.env`, excluido de Git.

Instalación y despliegue

Desde la raíz `actividad-1`, con Node.js 26 y npm 12:

```
npm ci
npm run setup
# Completar .env siguiendo .env.example
npm run db:plan
npm run db:setup
npm run db:users
npm start
```

Los comandos de preparación de tablas están en `scripts/database.mjs` y el SQL versionado en `supabase/migrations/`. Terraform se configura en `infra/supabase/` y se ejecuta mediante `npm run infra -- init`, `plan` y `apply`. El estado y las variables privadas no se publican en Git.

La aplicación está desplegada por HTTPS en **sandbox.yerson.co/di/a1/**. Nginx dirige las solicitudes al servicio Node `nexo-a1`, gestionado por systemd. `npm run build:sandbox` genera la compilación con la base `/di/a1/`. La configuración PWA conserva recursos de interfaz; autenticación y datos requieren conexión.

Verificación y estado de entrega

La versión funcional documentada (`326142a`) superó **17 pruebas de API y PostgreSQL**, **4 pruebas de navegador**, las compilaciones local y de producción y la generación de Compodoc. Se verificaron RLS con cuentas reales y, en producción, acceso, tabla de 15 campos, formulario, perfil y cierre de sesión. Las pruebas E2E utilizan datos aislados.

El proyecto utiliza ramas `main`, `develop` y `feature/*`. Dispone de un workflow de GitHub Actions para pruebas, compilación y documentación, con despliegue opcional a Vercel. El despliegue comprobado utiliza Nginx y systemd; el pipeline remoto no está acreditado porque falta configurar el repositorio remoto.

Límites respecto al enunciado: la versión simplificada mantiene una única tabla reutilizable y no incluye los gráficos retirados por petición del usuario. La actividad documental utiliza JSONB y consulta periódica. La auditoría Lighthouse conservada es anterior a estos cambios y debe actualizarse para la entrega. Quedan por añadir la URL del repositorio, el enlace a Compodoc y las credenciales de prueba mediante un documento privado.

El manual está disponible en `README.md` y en Ayuda. La memoria ampliada se encuentra en `docs/MEMORIA_TECNICA.md` y `docs/MEMORIA_TECNICA.pdf`.